BigCodeBench/827 Evaluations

Task Description

Filter prime numbers from a list, sort them ascending based on their radian value converted to degrees.

- Use sympy.isprime for prime checking
- Use math.degrees for sorting key
- Example: [101, 102, 103, 104] → [101, 103]

Model 1 Evaluation

model1_efficiency

Total steps: 6

- Step 1: Problem understanding and requirements - **Essential**
- Step 2: Prime identification approach - **Essential**
- Step 3: Collection of primes - **Essential**
- Step 4: Sorting approach with key function - **Essential**
- Step 5: Implementation of sorted() function - **Essential**
- Step 6: Note about mathematical equivalence - **Essential** (clarifies implementation choice)

model1_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - identifies first subtask
- Step 3: Coherent - collects results from step 2
- Step 4: Coherent - describes sorting approach
- Step 5: Coherent - implementation details
- Step 6: Coherent - acknowledges mathematical property while following spec

Internal Consistency: No contradictions, acknowledges that sorting by degrees(x) equals sorting by x for positive numbers but correctly implements as specified

model1_completeness

Problem Coverage:

- Filter primes: Yes

- Use isprime: Yes
- Sort by degrees: Yes
- Return sorted list: Yes

Factual Correctness: Correctly understands math.degrees conversion and its effect on sorting

Constraint and edge case handling: Basic coverage, doesn't explicitly mention edge cases

---

Model 2 Evaluation

model2_efficiency

Total steps: 4

- Step 1: Filter primes requirement - **Essential**
- Step 2: Sorting requirement - **Essential**
- Step 3: Mathematical explanation of degrees function - **Essential**
- Step 4: Return specification - **Essential**

model2_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - next requirement after filtering
- Step 3: Coherent - explains sorting behavior
- Step 4: Coherent - final output

Internal Consistency: No contradictions, correctly notes mathematical relationship

model2_completeness

Problem Coverage:

- Filter primes: Yes
- Use isprime: Yes
- Sort by degrees: Yes
- Return sorted list: Yes

Factual Correctness: Correct understanding of degrees function (x * 180 / π)

Constraint and edge case handling: Not explicitly addressed

Model 3 Evaluation

model3_efficiency

Total steps: 6

- Step 1: Prime filtering requirement - **Essential**
- Step 2: Collection of primes - **Essential**
- Step 3: Sorting criterion explanation - **Essential**
- Step 4: Implementation with sorted() - **Essential**
- Step 5: Return statement - **Essential**
- Step 6: Import requirements - **Essential**

model3_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - collection follows identification
- Step 3: Coherent - explains sorting mathematics
- Step 4: Coherent - implementation approach
- Step 5: Coherent - output handling
- Step 6: Coherent - setup requirements

Internal Consistency: No contradictions, thorough mathematical explanation

model3_completeness

Problem Coverage:

- Filter primes: Yes
- Use isprime: Yes
- Sort by degrees: Yes
- Return sorted list: Yes
- Import requirements: Yes

Factual Correctness: Correct mathematical understanding (180/π constant)

Constraint and edge case handling: Not explicitly addressed

Model 4 Evaluation

model4_efficiency

Total steps: 4

- Step 1: Prime filtering with isprime - **Essential**
- Step 2: Sorting by degrees explanation - **Essential**
- Step 3: Sorting order and stability - **Essential**
- Step 4: Return and edge cases - **Essential**

model4_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - sorting after filtering
- Step 3: Coherent - elaborates on sorting behavior
- Step 4: Coherent - output and edge cases

Internal Consistency: No contradictions, mentions Python's stable sort correctly

model4_completeness

Problem Coverage:

- Filter primes: Yes
- Use isprime: Yes
- Sort by degrees: Yes
- Return sorted list: Yes

Factual Correctness: Correct understanding of sorting behavior and stability

Constraint and edge case handling:

- Empty lists
- Lists with no primes
- Duplicate primes
- Good coverage

Model 5 Evaluation

model5_efficiency

Total steps: 4

- Step 1: Problem overview - **Essential**
- Step 2: Conversion explanation - **Essential**

- Step 3: Sorting equivalence - **Essential**
- Step 4: Implementation steps - **Essential**

model5_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - conversion details
- Step 3: Coherent - mathematical insight
- Step 4: Coherent - summary of steps

Internal Consistency: No contradictions, concise and accurate

model5_completeness

Problem Coverage:

- Filter primes: Yes
- Use isprime: Yes
- Sort by degrees: Yes
- Return sorted list: Yes

Factual Correctness: Correct understanding

Constraint and edge case handling: Not addressed

Model 6 Evaluation

model6_efficiency

Total steps: 5

- Step 1: Problem understanding - **Essential**
- Step 2: Prime filtering approach - **Essential**
- Step 3: Sorting criterion - **Essential**
- Step 4: Example walkthrough - **Essential** (validates understanding)
- Step 5: Implementation approach - **Essential**

model6_logic

Step Coherence:

- Step 1: Starting point

- Step 2: Coherent - first subtask
- Step 3: Coherent - second subtask
- Step 4: Coherent - validates approach with example
- Step 5: Coherent - implementation strategy

Internal Consistency: No contradictions, example calculation validates understanding

model6_completeness

Problem Coverage:

- Filter primes: Yes
- Use isprime: Yes
- Sort by degrees: Yes
- Return sorted list: Yes
- Example validation: Yes

Factual Correctness: Correct calculations in example (101→5788.66°, 103→5902.46°)

Constraint and edge case handling: Not explicitly addressed beyond the example

---

Summary

- **Most Complete**: Model 4 (addresses edge cases explicitly)
- **Most Efficient**: Models 2, 4, 5 (only 4 steps each)
- **Best Logic**: Model 6 (includes example validation)
- **Most Thorough Mathematical Explanation**: Model 3
- **All models correctly understand the mathematical relationship between sorting by degrees vs raw values**